

EXERCISE 2:

1) Define an integer array. Declare an integer pointer and initialize it to point the first element of the array. Use pointer arithmetic to traverse the array and print the elements.

```
#include <stdio.h>

int main()

{

    int arr[10], n, i;

    int *p = arr;

    printf("Enter n: ");

    scanf("%d", &n);

    printf("Enter elements: ");

    for(i = 0; i < n; i++)

    {

        scanf("%d", arr + i);

    }

    printf("Array elements: ");

    for(i = 0; i < n; i++)

    {

        printf("%d ", *(arr + i));

    }

}
```

```
    return 0;

}
```

OUTPUT:

Enter n: 3

Enter elements: 10 20 30

Array elements: 10 20 30

2) Calculate string length using pointers.

```
#include <stdio.h>

int main()
{
    char str[100];

    int i = 0;

    char *p;

    printf("Enter a string: ");

    scanf("%s", str);

    p = str;

    while(*(p + i) != '\0')
    {
        i++;
    }

    printf("Length of string: %d", i);
```

```
    return 0;

}
```

OUTPUT:

Enter a string: Kavi

Length of string: 4

3) Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

```
#include <stdio.h>

void calc(int n1, int n2, int *s, int *p)

{

    *s = n1 + n2;

    *p = n1 * n2;

}

int main()

{

    int a, b, sum, prod;

    printf("Enter two numbers: ");

    scanf("%d %d", &a, &b);

    calc(a, b, &sum, &prod);

    printf("Sum: %d\n", sum);

    printf("Product: %d\n", prod);
```

```
    return 0;

}
```

OUTPUT:

Enter two numbers: 30 40

Sum: 70

Product: 1200

4) Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

```
#include <stdio.h>

#include <stdlib.h>

struct stud

{

    int rno;

    char name[20];

};

int main()

{

    struct stud *s, *ptr;

    int n, i;

    printf("Enter n: ");
```

```
scanf("%d", &n);

s = (struct stud *)malloc(n * sizeof(struct stud));

if(s == NULL)

{

    printf("Memory allocation failed");

    return 1;

}

ptr = s;

printf("---- Enter Student Details ----\n");

for(i = 0; i < n; i++)

{

    printf("Roll no: ");

    scanf("%d", &(ptr + i)->rno);

    printf("Name: ");

    scanf("%s", (ptr + i)->name);

}

printf("\n---- Displaying Student Details ----\n");

for(i = 0; i < n; i++)

{

    printf("\nStudent %d:\n", i + 1);

    printf("Roll no: %d\n", (ptr + i)->rno);
```

```
        printf("Name: %s\n", (ptr + i)->name);  
    }  
  
    free(s);  
  
    return 0;  
}
```

OUTPUT:

Enter n: 3

---- Enter Student Details ----

Roll no: 86

Name: Pradee

Roll no: 98

Name: Rohi

Roll no: 128

Name: Rajesh

---- Displaying Student Details ----

Student 1:

Roll no: 86

Name: Pradee

Student 2:

Roll no: 98

Name: Rohi

Student 3:

Roll no: 128

Name: Rajesh

5) Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

```
#include <stdio.h>

int add(int a, int b)

{

    return a + b;

}

int multiply(int a, int b)

{

    return a * b;

}

int main()

{

    int (*operation_ptr)(int, int);

    int num1 = 5, num2 = 4;

    int result;

    operation_ptr = add;

    result = operation_ptr(num1, num2);
```

```
printf("Addition result: %d\n", result);

operation_ptr = multiply;

result = operation_ptr(num1, num2);

printf("Multiplication result: %d\n", result);

return 0;

}
```

OUTPUT:

Addition result: 9

Multiplication result: 20

6) Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

```
#include <stdio.h>

int main()

{

    int num = 10;

    int *p;

    int **pp;

    p = &num;

    pp = &p;

    printf("num: %d\n", num);

    printf("Address of num: %p\n", (void *)&num);
```



```
printf("*p (value stored in num): %d\n", *p);

printf("p: %p\n", (void *)p);

printf("&p: %p\n", (void *)&p);

printf("**pp: %d\n", **pp);

printf("*pp: %p\n", (void *)*pp);

printf("pp: %p\n", (void *)pp);

printf("Address of pp: %p\n", (void *)&pp);

return 0;

}
```

OUTPUT:

num: 10

Address of num: 0x7ffead27830c

***p (value stored in num): 10**

p: 0x7ffead27830c

&p: 0x7ffead278300

****pp: 10**

***pp: 0x7ffead27830c**

pp: 0x7ffead278300

Address of pp: 0x7ffead2782f8

7) Sort an array of floating point numbers using selection sort.

#include <stdio.h>

```
#include <stdlib.h>

int main()

{

    float *arr, temp;

    int i, j, n, smallest;

    printf("Enter size: ");

    scanf("%d", &n);

    arr = (float *)malloc(n * sizeof(float));

    if(arr == NULL)

    {

        printf("Memory allocation failed");

        return 1;

    }

    printf("Enter elements: ");

    for(i = 0; i < n; i++)

    {

        scanf("%f", &arr[i]);

    }

    for(i = 0; i < n - 1; i++)

    {

        smallest = i;
```

```
    for(j = i + 1; j < n; j++)  
    {  
        if(arr[j] < arr[smallest])  
        {  
            smallest = j;  
        }  
    }  
  
    temp = arr[i];  
    arr[i] = arr[smallest];  
    arr[smallest] = temp;  
}  
  
printf("Sorted array: ");  
  
for(i = 0; i < n; i++)  
{  
    printf("%.2f ", arr[i]);  
}  
  
free(arr);  
  
return 0;  
}
```

OUTPUT:

Enter size: 4

Enter elements: 78 89 90 87

Sorted array: 78.00 87.00 89.00 90.00

8) Define a structure student with fields such as id, name and grade. Create an array of student structures and sort them based on different fields.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Student
```

```
{
```

```
    int id;
```

```
    char name[20];
```

```
    float grade;
```

```
};
```

```
int main()
```

```
{
```

```
    struct Student s[5], temp;
```

```
    int n, i, j, choice;
```

```
    printf("Enter number of students: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter student details:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
printf("\nStudent %d\n", i + 1);

printf("ID: ");

scanf("%d", &s[i].id);

printf("Name: ");

scanf("%s", s[i].name);

printf("Grade: ");

scanf("%f", &s[i].grade);

}

printf("\nSort by:\n");

printf("1. ID\n");

printf("2. Name\n");

printf("3. Grade\n");

printf("Enter your choice: ");

scanf("%d", &choice);

for(i = 0; i < n - 1; i++)

{

    for(j = i + 1; j < n; j++)

    {

        if((choice == 1 && s[i].id > s[j].id) ||

            (choice == 2 && strcmp(s[i].name, s[j].name) > 0) ||

            (choice == 3 && s[i].grade > s[j].grade))
```

```
        {  
            temp = s[i];  
            s[i] = s[j];  
            s[j] = temp;  
        }  
    }  
}  
  
printf("\nSorted Student Details:\n");  
  
for(i = 0; i < n; i++)  
{  
    printf("ID: %d Name: %s Grade: %.2f\n",  
        s[i].id, s[i].name, s[i].grade);  
}  
  
return 0;  
}
```

OUTPUT:

Enter number of students: 3

Enter student details:

Student 1

ID: 86

Name: pradee

Grade: 9.11

Student 2

ID: 98

Name: rohi

Grade: 9.5

Student 3

ID: 128

Name: rajesh

Grade: 9.0

Sort by:

1. ID

2. Name

3. Grade

Enter your choice: 1

Sorted Student Details:

ID: 86 Name: pradee Grade: 9.11

ID: 98 Name: rohi Grade: 9.50

ID: 128 Name: rajesh Grade: 9.00

9) Sort a set of negative numbers using radix sort.

```
#include <stdio.h>
```

```
int getMax(int arr[], int n)
```

```
{  
  
    int max = arr[0];  
  
    int i;  
  
    for(i = 1; i < n; i++)  
  
    {  
  
        if(arr[i] > max)  
  
            max = arr[i];  
  
    }  
  
    return max;  
}  
  
void countingSort(int arr[], int n, int exp)  
  
{  
  
    int output[100];  
  
    int count[10] = {0};  
  
    int i;  
  
    for(i = 0; i < n; i++)  
  
        count[(arr[i] / exp) % 10]++;  
  
    for(i = 1; i < 10; i++)  
  
        count[i] = count[i] + count[i - 1];  
  
    for(i = n - 1; i >= 0; i--)  
  
    {
```



```

        output[count[(arr[i] / exp) % 10] - 1] = arr[i];

        count[(arr[i] / exp) % 10]--;

    }

    for(i = 0; i < n; i++)

        arr[i] = output[i];

}

void radixSort(int arr[], int n)

{

    int i, j, k = 0;

    int positive[100], negative[100];

    int p = 0, q = 0;

    for(i = 0; i < n; i++)

    {

        if(arr[i] < 0)

            negative[q++] = -arr[i];

        else

            positive[p++] = arr[i];

    }

    if(p > 0)

    {

        int max = getMax(positive, p);

```

```
        for(int exp = 1; max / exp > 0; exp *= 10)

            countingSort(positive, p, exp);

    }

    if(q > 0)

    {

        int max = getMax(negative, q);

        for(int exp = 1; max / exp > 0; exp *= 10)

            countingSort(negative, q, exp);

    }

    for(i = q - 1; i >= 0; i--)

        arr[k++] = -negative[i];

    for(j = 0; j < p; j++)

        arr[k++] = positive[j];

    }

int main()

{

    int arr[100], n, i;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    printf("Enter elements: ");
```

```
for(i = 0; i < n; i++)  
    scanf("%d", &arr[i]);  
  
radixSort(arr, n);  
  
printf("Sorted array: ");  
  
for(i = 0; i < n; i++)  
    printf("%d ", arr[i]);  
  
return 0;  
  
}
```

OUTPUT:

Enter number of elements: 3

Enter elements: 30 40 -3

Sorted array: -3 30 40

10) Write a C program to copy one array to another using pointers.

```
#include <stdio.h>  
  
int main()  
{  
  
    int arr1[100], arr2[100];  
  
    int n, i;  
  
    int *p, *q;  
  
    printf("Enter number of elements: ");  
  
    scanf("%d", &n);
```

```
printf("Enter elements of first array: ");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    scanf("%d", &arr1[i]);
```

```
}
```

```
p = arr1;
```

```
q = arr2;
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    *(q + i) = *(p + i);
```

```
}
```

```
printf("First array: ");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("%d ", *(p + i));
```

```
}
```

```
printf("\nCopied array: ");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("%d ", *(q + i));
```

```
}
```

```
    return 0;  
}
```

OUTPUT:

Enter number of elements: 3

Enter elements of first array: 30 40 50

First array: 30 40 50

Copied array: 30 40 50

